

Inteligencia Artificial Generativa Aplicada en Biomedicina

Módulo 6: Agents

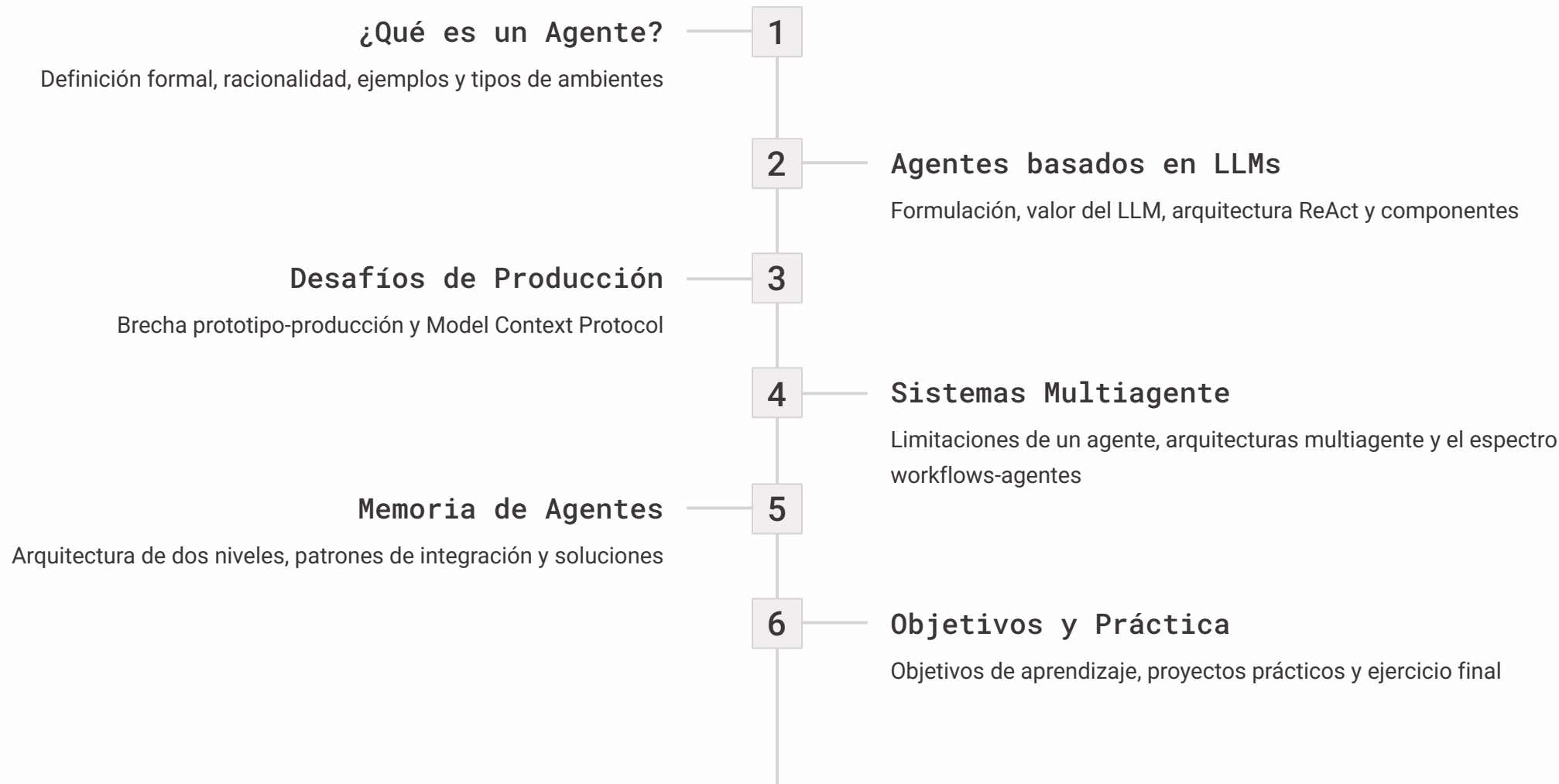
Profesores: Paulo Veiga / Marco Sanchez Sorondo

Universidad Austral · Facultad de Ingeniería · Departamento de
Inteligencia Artificial

Abril, 2026



Agenda



¿QUÉ ES UN AGENTE?

¿Qué es un **Agente**?

Antes de hablar de agentes basados en LLMs, necesitamos entender la definición formal y rigurosa del concepto.

- ❏ En IA clásica, un agente es cualquier entidad que percibe su entorno mediante sensores y actúa sobre él mediante actuadores. — Russell & Norvig

Agentes – Una palabra mal utilizada

En el uso cotidiano, la palabra "agente" se aplica con demasiada frecuencia a sistemas basados en LLMs, pero esa simplificación suele ocultar distinciones importantes.



¿Qué se suele llamar agente?

En el uso popular, se denomina "agente" simplemente a un LLM o a cualquier sistema basado en LLM, sin más distinción.



¿Por qué esto es incorrecto?

Un LLM no es un agente por sí solo. Además, existen muchos tipos de agentes que no involucran en absoluto modelos de lenguaje.



¿Cuál es el camino correcto?

Entender la definición formal de agente antes de hablar de agentes basados en LLMs es indispensable para razonar con rigor.

¿QUÉ ES UN AGENTE?

"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators..."

— Russell & Norvig, *Artificial Intelligence: A Modern Approach*

Esta es la definición canónica y punto de partida de toda la teoría de agentes inteligentes en IA clásica.

¿QUÉ ES UN AGENTE?

Comentarios sobre la definición

La elegancia de esta definición radica en su **amplitud y generalidad**. Veamos lo que *no* menciona:



Sin ML ni LLMs

No se mencionan Machine Learning, Deep Learning, LLMs, tools ni RAG. Son implementaciones posibles, no la definición.



Sin programas

Ni siquiera se mencionan programas o software. Un sistema mecánico o electrónico también encaja perfectamente.



Sistemas biológicos

La definición contempla sistemas biológicos: células, humanos, animales e insectos son todos agentes válidos bajo esta definición.

"A rational agent is one that does the right thing... This notion of desirability is captured by a performance measure that evaluates any given sequence of environment states."

— Russell & Norvig, *Artificial Intelligence: A Modern Approach*

La racionalidad introduce el concepto de **optimización**: no basta con percibir y actuar, hay que hacerlo bien según un criterio definido.

¿QUÉ ES UN AGENTE?

Racionalidad

Un **agente racional** selecciona la acción que maximiza su medida de performance, dada la evidencia de sus percepciones y su conocimiento previo. La racionalidad depende de cuatro factores clave:



Medida de performance

El criterio que define el éxito del agente. Sin esta medida, no hay racionalidad posible.



Conocimiento previo

Lo que el agente ya sabe sobre el ambiente antes de comenzar a actuar.



Acciones disponibles

El conjunto de acciones que el agente puede ejecutar en su entorno.



Secuencia de percepciones

El historial de todo lo que el agente ha observado del ambiente hasta el momento.

¿QUÉ ES UN AGENTE?

Ejemplo: Vacuum Cleaner

Formulación como agente inteligente — Mantener limpio un entorno (grilla $n \times m$) eliminando la suciedad con el mínimo de acciones posibles.

Agente	Aspiradora autónoma (robot o modelo abstracto)
Sensores	Detector de suciedad, posición actual en la grilla
Actuadores	Motor de movimiento (izq/der/adelante/atrás), motor de succión
Ambiente	Grilla $n \times m$, parcialmente observable, determinístico
Acciones	Aspirar, MoverIzquierda, MoverDerecha, Esperar
Performance	+10 limpiar celda sucia · -1 moverse · -5 aspirar celda limpia · -10 chocar

¿QUÉ ES UN AGENTE?

Ejemplo: Robot Móvil

Formulación como agente inteligente — Desplazarse desde un punto inicial hasta un destino evitando obstáculos en entornos complejos.

Agente	Robot móvil autónomo con sistema de control y planificación
Sensores	Cámara, LiDAR, GPS, giroscopio, sensores de proximidad
Actuadores	Motores de tracción, dirección, frenos, brazo robótico
Ambiente	Entorno físico (indoor/outdoor), continuo, parcialmente observable, dinámico
Acciones	Avanzar, girar, frenar, recalcular ruta
Performance	Llegar al destino rápido, sin colisiones y con mínimo consumo de energía

¿QUÉ ES UN AGENTE?

Ejemplo: DeepMind AlphaGo

Formulación como agente inteligente — Jugar al Go a nivel superhumano mediante redes neuronales profundas y búsqueda Monte Carlo.

Agente	Red neuronal profunda + búsqueda Monte Carlo (MCTS)
Sensores	Estado actual del tablero (disposición de fichas blancas y negras)
Actuadores	Colocar una piedra en una intersección válida o pasar turno
Ambiente	Tablero de Go 19×19, determinístico, completamente observable, competitivo
Performance	Probabilidad de victoria en la partida

¿QUÉ ES UN AGENTE?

De AlphaGo Zero a AlphaZero

La evolución de AlphaGo ilustra uno de los saltos más significativos en la historia de los agentes inteligentes: pasar del aprendizaje con datos humanos a la **generalización pura**.



AlphaGo

Aprende de partidas humanas. Requiere datos de expertos como punto de partida.



AlphaGo Zero

Aprende desde cero sin datos humanos, solo autojuego y retroalimentación. Red neuronal unificada política-valor + MCTS.



AlphaZero

Generaliza a múltiples juegos (ajedrez, shogi, go) sin conocimiento previo. Mismo algoritmo, distintos juegos.



La métrica de performance es universal: tasa de victorias (ganar > empatar > perder).

Tipos de Ambientes

La naturaleza del ambiente determina en gran medida la complejidad del agente necesario para operar en él. Estas son las dimensiones fundamentales de clasificación:



Observable

Completamente observable vs.
Parcialmente observable



Agentes

Un agente vs. Multi-agente



Determinismo

Determinístico vs. Estocástico



Episódico vs. Secuencial

Acciones independientes vs. acciones
con consecuencias a largo plazo



Estático vs. Dinámico

El ambiente cambia o no mientras el
agente delibera



Discreto vs. Continuo

Estados y acciones finitos vs. infinitos



Conocido vs. Desconocido

El agente conoce o no las reglas del
ambiente

Dificultad de Ambientes

Las dimensiones del ambiente se combinan para determinar cuán desafiante resulta operar en él. El espectro va de problemas triviales a extraordinariamente complejos.

1

Crucigrama

Fácil: observable, determinístico, estático. Un solo agente.

2

Ajedrez

Medio: observable y determinístico, pero competitivo. Requiere anticipar al oponente.

3

Taxi Autónomo

Difícil: parcialmente observable, estocástico, dinámico, continuo, multi-agente.

¿QUÉ ES UN AGENTE?

¿Por qué todas estas definiciones?

La formalización agentica no es un ejercicio académico vacío. Tiene tres utilidades prácticas concretas y poderosas:



Análisis riguroso

Nos obliga a pensar bien en el problema a resolver y analizar todos sus aspectos antes de implementar.



Reutilización de frameworks

Permite usar frameworks agnósticos que resuelven la formulación agentica. Un mismo algoritmo puede ganar en ajedrez y dirigir un robot.



Decisión informada

Es fundamental saber cuándo vale la pena usar un agente y cuándo una solución más simple es suficiente.

¿QUÉ ES UN AGENTE?

¿Cuándo vale la pena usar agentes?

Los agentes no son la solución a todo. Estas son las condiciones que justifican su uso frente a soluciones determinísticas más simples:

→ Múltiples herramientas en orden variable

Cuando la solución requiere usar varias herramientas, en múltiples órdenes y de distintas maneras según el contexto.

→ Interpretación dinámica de resultados

Cuando hay que interpretar y relacionar apropiadamente resultados en tiempo de ejecución, adaptando el comportamiento.

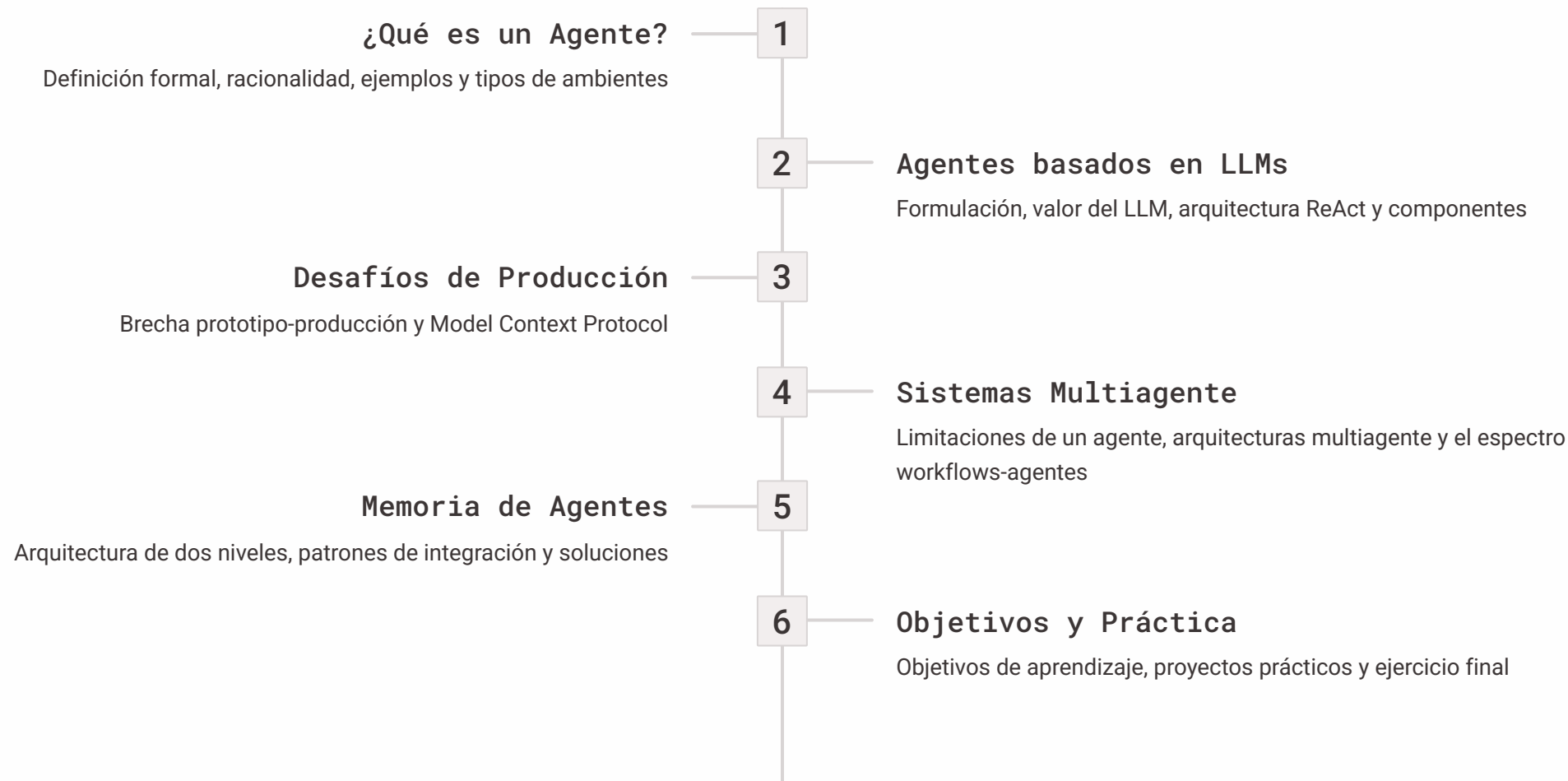
→ Orquestación variable de sistemas

Cuando hay que sincronizar y/u orquestar distintos sistemas de manera variable según las circunstancias.

→ Solución determinística insuficiente

Cuando el espacio de posibilidades es demasiado grande o dinámico para ser cubierto con reglas fijas.

Agenda



AGENTES BASADOS EN LLMS

Agentes basados en LLMs

Con la base teórica asentada, exploramos cómo los modelos de lenguaje encajan en el marco formal de agentes inteligentes.

Agentes basados en LLMs

Formulación como agente inteligente — Resolver tareas lingüísticas o de razonamiento a partir de instrucciones en lenguaje natural.

Agente	Modelo de lenguaje grande (LLM) con razonamiento, memoria y acceso a herramientas externas
Sensores	Texto de entrada del usuario, contexto previo, resultados de herramientas
Actuadores	Generar texto, razonar, ejecutar comandos, llamar APIs, producir planes
Ambiente	Entorno digital, simbólico, parcialmente observable, dinámico
Performance	Calidad, relevancia y precisión de respuestas; satisfacción del usuario

¿Cuál es el valor de un LLM en un agente?

El LLM no es simplemente un chatbot dentro de un agente. Su valor radica en capacidades genuinamente novedosas que ningún sistema anterior podía ofrecer:



Observar el lenguaje natural

Abre las puertas a un agente genérico para "percibir" el ambiente del lenguaje natural y actuar en él de forma fluida.



Reflexionar sobre outputs

Puede analizar el resultado de una herramienta (ej: query con error → corrige → reintenta) en un ciclo reflexivo.



Usar herramientas con lenguaje

Emplea el lenguaje natural como medio universal para interactuar con herramientas de cualquier tipo y API.

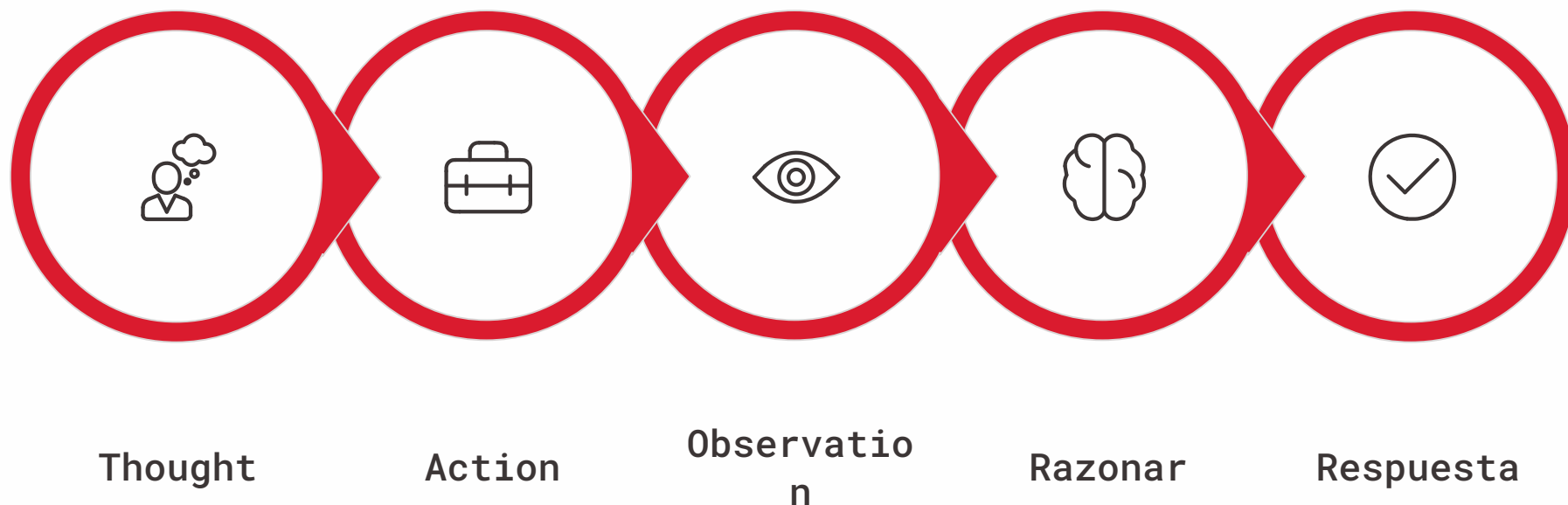


Instruir sistemas complejos

Permite comunicarse con sistemas informáticos complejos usando lenguaje desestructurado y natural.

Arquitectura **ReAct** (Reason + Act)

ReAct es la arquitectura más simple y fundamental para agentes basados en LLMs. Combina razonamiento y acción en un ciclo iterativo hasta alcanzar una respuesta final.



El ciclo **Thought** → **Action** → **Observation** se repite hasta que el agente determina que tiene suficiente información para producir una respuesta final de calidad.



AI Agents

Introduction to AI Agents

What agents are, the ReAct loop, and building your first agent with LangChain tool calling



AI Tutorial



Introduction to AI Agents – AI Tutorial

What agents are, the ReAct loop, and building your first agent with LangChain tool calling

Componentes de un Agente basado en LLM

Todo agente basado en LLM se construye sobre tres componentes esenciales que definen qué sabe, qué puede hacer y de qué contexto dispone:



1. Memoria y Contexto

Historial de conversación, último prompt y contexto activo. Es la "conciencia" inmediata del agente sobre la situación actual.



2. Tools (Herramientas)

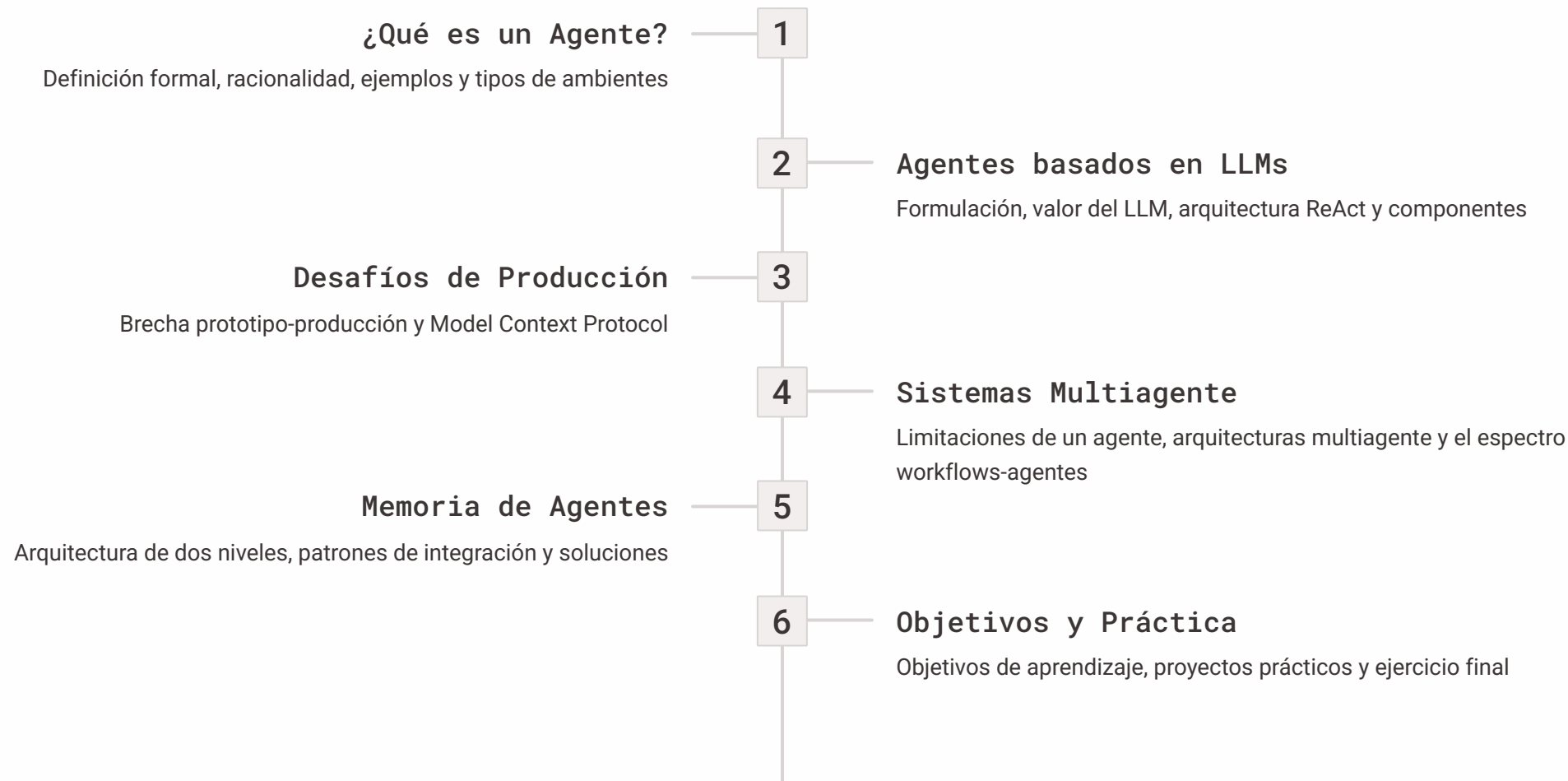
Conjunto de herramientas a las que puede llamar, descritas en el contexto. El agente decide cuándo y cómo invocarlas.



3. Información Contextual (RAG)

Información asociada al historial mediante recuperación semántica. Puede incrustarse en el prompt o invocarse como herramienta.

Agenda



DESAFÍOS DE PRODUCCIÓN

Desafíos de Producción

Llevar agentes del prototipo funcional a un sistema de producción confiable es uno de los mayores retos de la ingeniería de IA actual.

¿Por qué importa en producción?

Existe una brecha significativa entre un demo que funciona y un sistema de producción confiable. Los datos son reveladores:

95%+

Accuracy requerida

El umbral mínimo para sistemas de producción confiables. Los demos funcionales no llegan a este nivel.

58%

Tool selection con 20+ tools

La accuracy de selección de herramientas cae de 92% (5 tools) a solo 58% con más de 20 herramientas disponibles.

~85%

Rule enforcement

Los LLMs cumplen reglas basadas en prompt el 85% del tiempo. Insuficiente para sectores regulados.

❑ Las fallas en producción vienen de tool design inadecuado, estructuras de memoria y mecanismos de validación — no del modelo en sí.

MCP: Model Context Protocol

El **Model Context Protocol** es el estándar emergente de integración para agentes, diseñado para resolver la fragmentación del ecosistema y garantizar interoperabilidad a largo plazo.

Estándar universal

Asegura compatibilidad entre Claude, ChatGPT, Cursor y futuras plataformas. Diseña una vez, integra en todas partes.

Servidores MCP

Permite diseñar y operar servidores MCP con especificaciones completas de herramientas, recursos y capacidades.

Multi-servidor

Los agentes pueden integrarse con múltiples servidores MCP simultáneamente via `MultiServerMCPClient`.

Ciclo estandarizado

Define cómo los agentes descubren, llaman y reciben resultados de herramientas de manera consistente.

Agenda

¿Qué es un Agente?

Definición formal, racionalidad, ejemplos y tipos de ambientes

1

2

Agentes basados en LLMs

Formulación, valor del LLM, arquitectura ReAct y componentes

Desafíos de Producción

Brecha prototipo-producción y Model Context Protocol

3

4

Sistemas Multiagente

Limitaciones de un agente, arquitecturas multiagente y el espectro workflows-agentes

Memoria de Agentes

Arquitectura de dos niveles, patrones de integración y soluciones

5

6

Objetivos y Práctica

Objetivos de aprendizaje, proyectos prácticos y ejercicio final

SISTEMAS MULTIAGENTE

Sistemas Multiagente

Cuando un solo agente no alcanza: motivación, arquitecturas y aplicación en biomedicina.

Limitaciones de un solo agente

Ya vimos que un agente con tools puede resolver problemas complejos. Pero a medida que crece el número de herramientas y dominios, aparecen problemas concretos:



Sobrecarga de tools

La accuracy de selección de herramientas cae drásticamente cuando el agente tiene demasiadas opciones (de ~92% con 5 tools a ~58% con 20+). Un agente generalista pierde precisión.



Contexto desbordado

Si el agente debe manejar información clínica, farmacológica, radiológica y administrativa en un solo contexto, la calidad de razonamiento se degrada.



Falta de especialización

Un único system prompt no puede ser simultáneamente experto en interacciones farmacológicas Y en interpretación de ensayos clínicos Y en análisis de imágenes.

Ejemplo: Soporte a Decisiones Clínicas

Queremos asistir a un equipo médico en la toma de decisiones para un paciente con cáncer de pulmón (NSCLC), integrando datos clínicos, farmacológicos y de evidencia.

Tools 15+ tools mezcladas de todos los dominios 3–5 tools por agente, especializadas System prompt Genérico, intenta cubrir todo Cada agente tiene instrucciones de su dominio Contexto Se llena rápidamente con info de todos los dominios Cada agente mantiene solo su contexto relevante Precisión Se degrada con la complejidad Cada agente es experto en lo suyo

El problema no es la capacidad del LLM, sino la arquitectura. Dividir en agentes especializados es una decisión de ingeniería, no de modelo.

¿Qué es un Sistema Multiagente?

Un sistema multiagente consta de múltiples agentes que interactúan entre sí — de forma colaborativa, competitiva, o ambas — para resolver un problema que excede las capacidades de un agente individual.



Modularidad

Agentes independientes facilitan el desarrollo, testing y mantenimiento. Se puede mejorar un agente sin tocar los demás.



Especialización

Cada agente tiene su propio dominio de expertise, sus tools y su system prompt optimizado para una tarea concreta.



Control

Se define explícitamente cómo se comunican los agentes y quién decide el flujo, en vez de depender de un único LLM para todo.

Ya vimos ejemplos multiagente: AlphaGo (competitivo), Taxi Autónomo (competitivo + colaborativo). En biomedicina, el patrón típico es colaborativo.

Arquitecturas Multiagente

Hay varias formas de conectar agentes. La elección depende del nivel de control, complejidad y autonomía que necesite el sistema.



Supervisor

Un agente central decide a qué agente especializado derivar cada consulta. Patrón más común y predecible. Ejemplo: un agente coordinador recibe la pregunta del médico y la deriva al agente clínico o al farmacológico según corresponda.



Supervisor (tool-calling)

Los agentes especializados se exponen como tools del supervisor. El LLM supervisor usa function calling para invocarlos. Ejemplo: es lo que los alumnos van a implementar en la Parte 3 del proyecto.



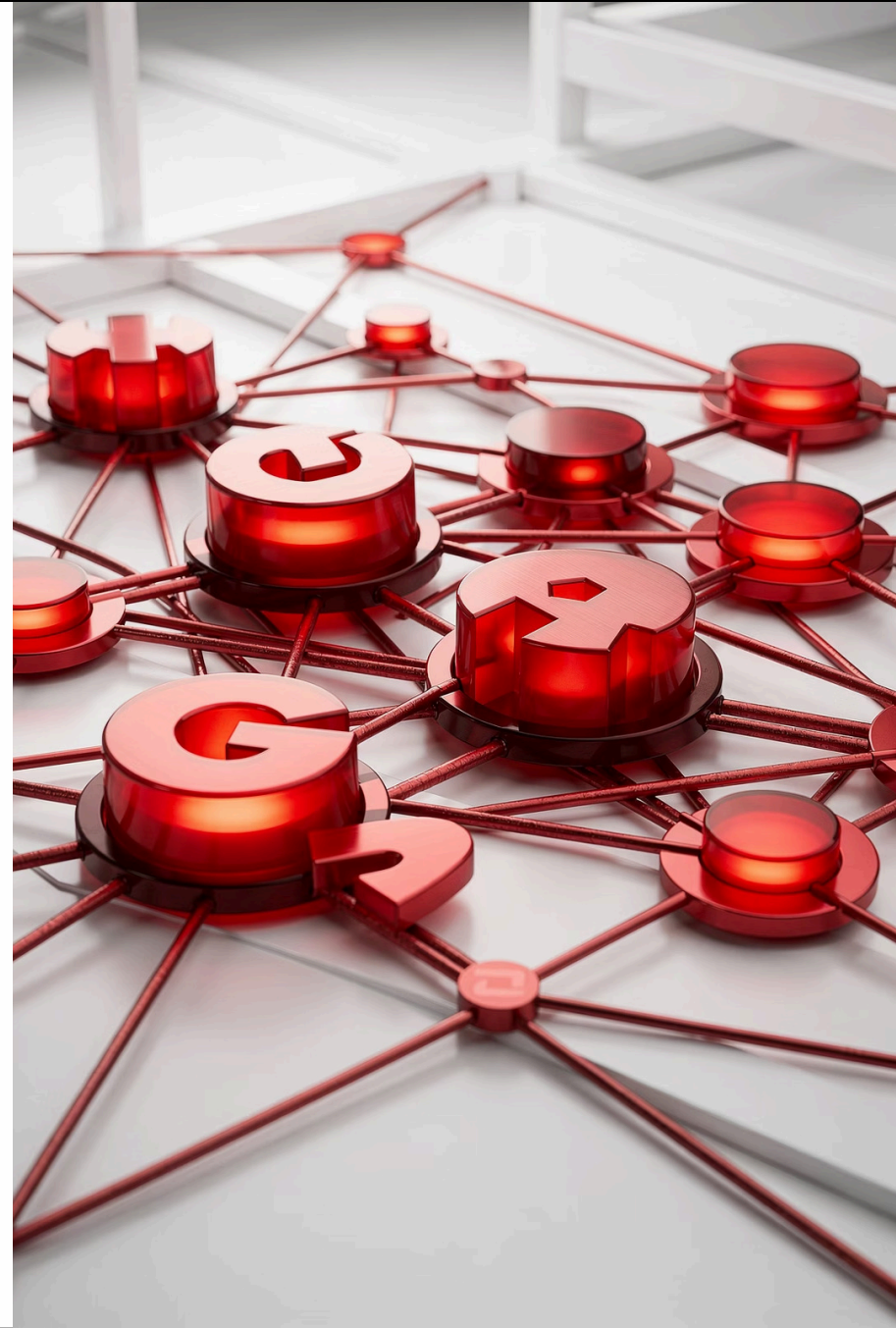
Network

Todos los agentes se comunican entre sí. Más flexible pero menos predecible. Ejemplo: agente clínico y farmacológico dialogan directamente para resolver una interacción.



Hierarchical

Supervisor de supervisores. Para sistemas muy complejos. Ejemplo: hospital con departamentos, cada uno con su coordinador interno.



Workflows vs Agentes – El espectro

No todo sistema con múltiples LLMs es un 'agente'. Existe un espectro entre flujos deterministas y agentes autónomos.

Workflows (deterministas)

Rutas de código predefinidas. El LLM se embebe en pasos fijos.

Patrones: prompt chaining, paralelización.

Ejemplo: Pipeline que siempre ejecuta
análisis clínico → verificación
farmacológica → informe.

Workflows (dirigidos por LLM)

El LLM dirige el flujo entre rutas predefinidas.

Patrones: routing, orchestrator-worker, evaluator-optimizer.

Ejemplo: Un router decide si la pregunta va al agente clínico o al farmacológico.

Agentes (autónomos)

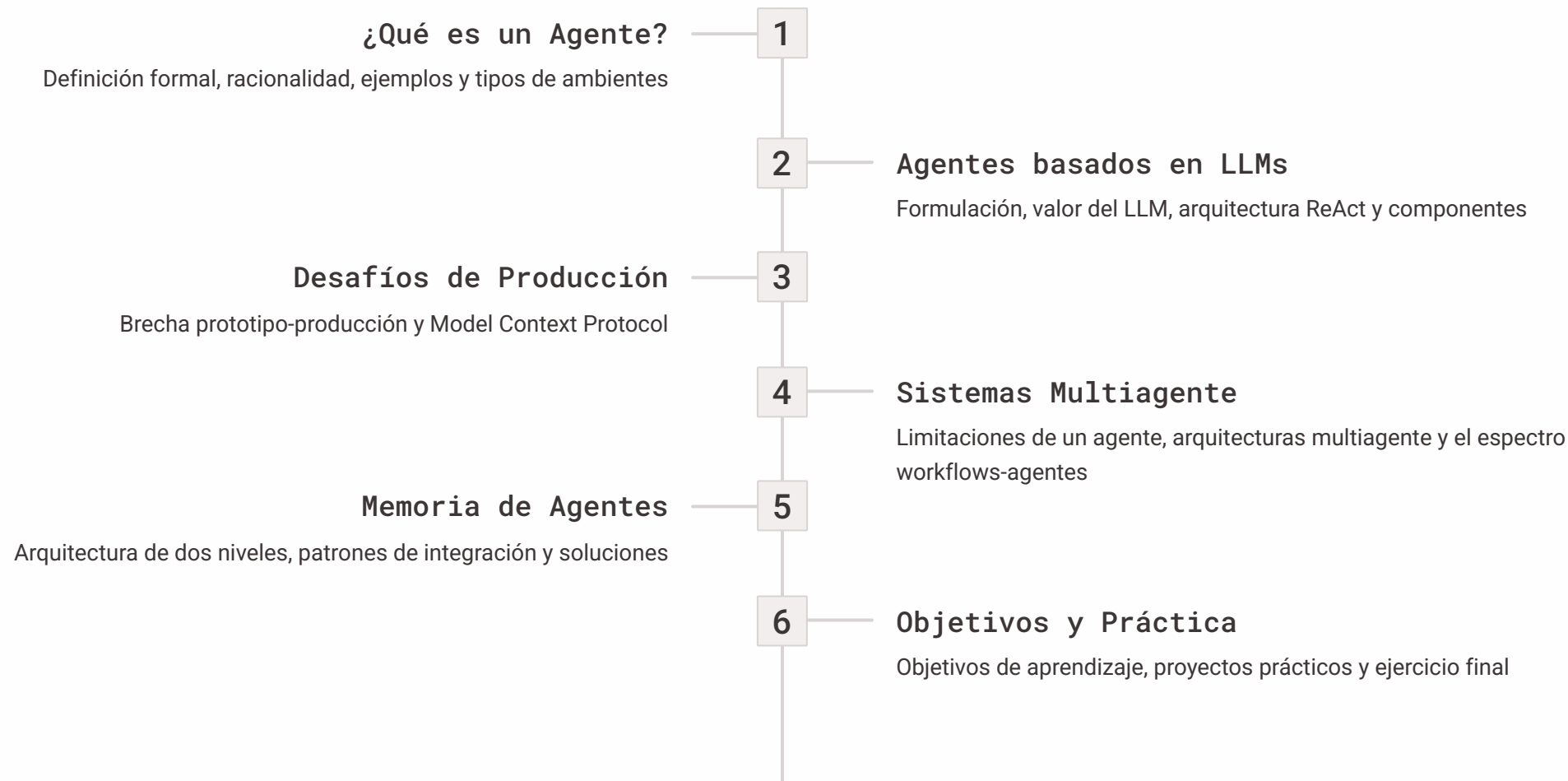
El LLM decide sus propias acciones basado en feedback del ambiente.

Patrón: ReAct con tool-calling.

Ejemplo: El agente decide qué tools usar, en qué orden, y cuándo tiene suficiente información para responder.

La magia está en mezclarlos. Un sistema real puede tener routing determinista entre agentes que internamente son autónomos.

Agenda



MEMORIA DE AGENTES

Memoria de Agentes

La memoria es uno de los componentes más críticos y menos intuitivos de los agentes en producción. Sin una estrategia adecuada, los agentes son caros, incoherentes y frustrantes.

Arquitectura de Memoria: Dos Niveles

La solución al problema de memoria es una arquitectura que combina velocidad con persistencia, inspirada en los sistemas de caché de hardware:

Working Memory (Sesión)

Duración: Una sola sesión activa

Propósito: Estado activo de conversación

Ejemplo: Detalles del pedido en el chat actual

Implementación: MemorySaver + thread_id

Analogía: Caché L1 — rápida y temporal

Long-Term Memory (Persistente)

Duración: Entre sesiones, indefinidamente

Propósito: Conocimiento persistente del usuario

Ejemplo: Preferencias de usuario a lo largo de meses

Implementación: Vector DB o servicio gestionado

Analogía: Base de datos — persistente y buscable

Patrones de Integración de Memoria

Existen tres patrones principales para integrar la memoria en un agente, con distintos niveles de control y autonomía:

01

Code-Driven (Programático)

Tu código decide explícitamente cuándo guardar y recuperar. Comportamiento predecible y eficiente. Punto de partida recomendado.

02

LLM-Driven (Tool-Based)

El agente recibe tools de memoria y decide autónomamente qué recordar. Más natural y flexible, pero menos predecible.

03

Background Extraction (Automático)

Almacena toda la conversación; procesos en segundo plano extraen hechos importantes de forma asíncrona.

📌 **Recomendación:** Empezar con code-driven, agregar background extraction para enriquecimiento, y usar LLM-driven cuando la autonomía sea un requisito funcional.



AI Agents

Agent Memory

Working memory with MemorySaver and long-term
memory for cross-session persistence



AI Tutorial



Agent Memory – AI Tutorial

Working memory with MemorySaver and long-term memory for cross-session persistence

Soluciones de Memoria en Producción

El ecosistema de memoria para agentes ha madurado rápidamente. Estas son las soluciones más relevantes y el patrón universal que todas implementan:

Redis Agent Memory Server

Working + long-term con búsqueda semántica integrada. Alta performance para producción.

Mem0

Capa de memoria gestionada para agentes. API simple, ideal para integración rápida.

Zep

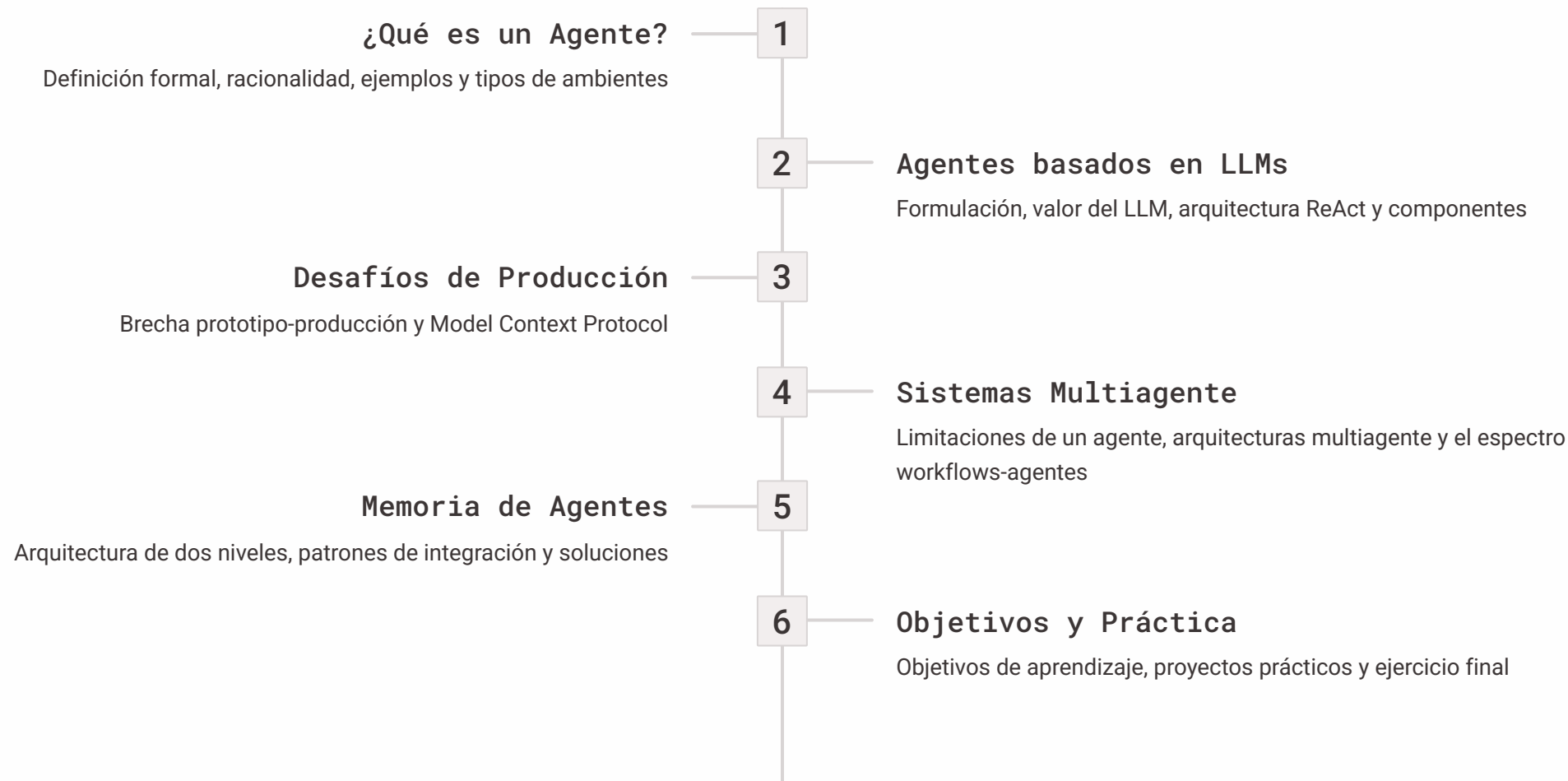
Long-term memory con extracción automática de hechos desde conversaciones.

LangChain Memory

Integración nativa con LangChain/LangSmith. La opción más directa si ya usás el stack de LangChain.

📌 **Patrón universal:** store → search → inject en prompt o tool results. Independientemente de la solución elegida, este flujo es constante.

Agenda



OBJETIVOS Y PRÁCTICA

Objetivos y Práctica

Esta parte del módulo traduce toda la teoría en competencias concretas y proyectos prácticos implementables.

Objetivos de Aprendizaje

Al finalizar este módulo, el estudiante será capaz de diseñar, implementar y operar agentes basados en LLMs en entornos de producción:



Construir agentes con tool capabilities

Usando `createAgent` de LangChain con patrones ReAct completos.



Diseñar y operar servidores MCP

Con especificaciones completas e integración multi-servidor via `MultiServerMCPClient`.



Implementar memoria y seguridad

Memoria con `MemorySaver`, reglas de negocio determinísticas, detección de PII y mitigación de jailbreak.

Ejercicio Práctico

El siguiente ejercicio busca integrar todos los conceptos del módulo en un problema original elegido por cada estudiante:

1

Paso 1: Elegir un problema

Pensar en un problema real a ser resuelto por un agente basado en LLMs. Cuanto más cercano a la realidad, mejor.

2

Paso 2: Modelar como agente

Formularlo con las definiciones formales: agente, sensores, actuadores, ambiente y medida de performance.

3

Paso 3: Implementar

Resolverlo usando como base la notebook de la práctica. Se puede usar información hardcodeada (JSONs, CSVs, .txt, etc.).

Guía de ejercicios: <https://aitutorial.dev/agents/hands-on-exercise>

Bibliografía y Recursos

Recursos fundamentales para profundizar en los conceptos de este módulo:

Russell & Norvig (2020)

Artificial Intelligence: A Modern Approach, 4th ed. Pearson. La referencia canónica para la teoría formal de agentes inteligentes.

LangGraph Docs

Documentación oficial del framework para construcción de agentes stateful con grafos. langchain-ai.github.io/langgraph

AI Tutorial – Agents Module

Guía práctica sobre el módulo de agentes con ejemplos de código. aitutorial.dev/agents/overview

LangChain Memory

Documentación de los módulos de memoria de LangChain para agentes. python.langchain.com/docs/modules/memory